

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Воронежский государственный технический университет»

Кафедра систем управления и информационных технологий в строительстве

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ
по курсовому проектированию по дисциплине
«ТЕХНОЛОГИЯ ПРОГРАММИРОВАНИЯ»

для обучающихся по направлению
09.03.02 «Информационные системы и технологии»
всех форм обучения

(Электронный ресурс)

Воронеж 2020

УДК 004.4
ББК 32.973. 3я73
Составители О.В. Минакова

Методические рекомендации по выполнению лабораторных работ по дисциплине «Технология программирования» для обучающихся по направлению 09.03.02 «Информационные системы и технологии», всех форм обучения / ФГБОУ ВО «Воронежский государственный технический университет»; сост.: О.В. Минакова. – Воронеж: Изд-во ВГТУ, 2020. – 33 с.

Представлен порядок выполнения, шаблоны оформления программных артефактов и рекомендации по разработке программного продукта в ходе курсового проектирования по дисциплине «Технология программирования».

Предназначены для студентов направления 09.03.02 «Информационные системы и технологии» всех форм обучения.

УДК 004.4
ББК 32.973.3я73

Рецензент -

ВВЕДЕНИЕ

Для уменьшения стоимости разработки программ и повышения производительности труда программистов используются методы, регламентирующие высокую профессиональную культуру написания программ независимо от языка, от системы, от ЭВМ и решаемой задачи. Такие методы получили общее название – технологии программирования. В соответствии с значением слова «технология» под технологией программирования следует понимать совокупность производственных процессов, приводящую к созданию требуемого программного средства (ПС), а также описание этой совокупности процессов.

Хорошая технология дает возможность получить высокий экономический эффект при ее использовании, существенный рост производительности труда программистов, повышает качество программного продукта. Современный IT-инженер должен знать основные этапы и принципы создания программного продукта, механизмы абстракции, рекурсии, типизации, повторного использования, различие между спецификацией и реализацией, конфиденциальность информации, понимать проблему сложности, масштабирования, проектирование с учетом изменений, классификация, соглашения, обработку исключений, уметь разрабатывать информационно-логическую, функциональную и объектно-ориентированную модели программных средств, модели данных программных средств, а также разрабатывать, согласовывать и выпускать все виды документации программных средств.

Цель курсовой работы – достижение студентами владения одной из технологий программирования и использования инструментальных средств разработки программного обеспечения. Это предполагает самостоятельное проведение всех этапов разработки программного средства, включающее обоснование выбора структурной и функциональной схемы программного обеспечения, структур данных, алгоритмов, стратегии тестирования и отладки программного обеспечения.

Важнейшей частью курсовой работы является составление пояснительной записки, содержащей обоснование принятых проектных решений и применение нормативных документов, регламентирующих состав, содержание и форму технической документации на разработанный программный продукт.

1. Организация процесса выполнения курсового проекта

1. 1. Порядок выполнения проекта

Работа выполняется студентами по индивидуальным заданиям. Заданием для курсового проекта является исходная формулировка задачи или проблемы, в терминах предметной области.

Примеры тем.

1. *Разработка программного средства учета личных затрат на коммунальные услуги.*

2. *Разработка интерактивной обучающей программы по теме «Кодирование чисел в ЭВМ».*

3. *Разработка программного обеспечения домашней мультимедийной библиотеки.*

4. *Разработка программной системы расчета простейших электрических схем.*

На основе исходной формулировки проблемы или существующей потребности студент должен сделать уточненную постановку задачи и согласовать ее с научным руководителем. Это совместное соглашение оформляется в виде технического задания (ТЗ) и размещается в приложении к пояснительной записке. Выбор алгоритма программы осуществляется произвольно, допустимо любое стандартное или авторское решение, со ссылкой на источник информации, содержащий методико-информационное обеспечение для составления алгоритма.

В процессе выполнения курсовой работы студенты должны:

1. разработать развернутое техническое задание на программный продукт;
2. выполнить анализ задания, выбрать технологию проектирования и разработать проект программного продукта (логическую и физическую модели с учетом динамических аспектов);
3. выбрать структуры данных (модель) для реализации предметной области программного продукта;
4. разработать интерфейс пользователя;
5. выбрать стратегию тестирования и разработать тестовые наборы;
6. выбрать язык и среду программирования;
7. предложить архитектуру ПС, разделить функциональность по компонентам;
8. выбрать паттерны проектирования для решения и реализовать их в выбранной среде разработки;
9. выполнить тестирование и отладку;
10. разработать необходимую документацию, указанную в техническом задании.

Результатом курсового проектирования является программа на языке высокого уровня и совокупность разработанных текстовых и графических документов – пояснительная записка с **документацией** процесса разработки ПО, включающей:

- концепцию программного проекта;
- техническое задание на проектирование или спецификацию программ-

ного средства;

- диаграмму классов разработанного приложения;
- лист верификации;
- руководство пользователя;
- оформленный по установленным правилам код программы (стиль оформления в соответствии со стандартами кодирования выбранного языка программирования, комментарии для автоматической компиляции).

По результатам выполнения работы оформляется пояснительная записка, включающая введение, разделы и подразделы основной части, заключение, список использованных источников, приложения.

Во введении необходимо определить необходимость разработки заданной программы и сформулировать задачи, которые должны быть решены в рамках проектирования.

Основная часть состоит из трех разделов, в которых описываются основные этапы проектирования, включающие:

1. Составление внешнего описания (спецификации) программного продукта.

2. Проектирование ПС, включая описание архитектуры, ее детализация и выбор моделей для реализации, включая проектирования графического интерфейса.

3. Описание процесса реализации, тестирования и размещения программного продукта.

Заключение должно содержать основные результаты работы и оценку соответствия полученной программы функциональным и нефункциональным требованиям.

В приложении следует представить документацию процесса разработки и самого продукта.

На защиту студент предоставляет:

- техническое задание;
- программное средство;
- пояснительную записку, содержащую описание этапов разработки ПС и соответствующие иллюстрации, и разработанную программную документацию.

На защите курсового проекта студент коротко (2–3 мин.) докладывает об основных проектных решениях, принятых в процессе разработки, и отвечает на вопросы.

Оценка работы складывается из следующих составляющих:

- 1 – внешнее описание должно иметь 3 части – общие требования, спецификация качества и функциональные требования с описанием особых случаев работы программы;

- 2 – представлена архитектура ПС, выполнено функциональное разделение по модулям, обосновано проектирование интерфейса пользователя и модели данных;

- 3 – обоснован выбор паттернов проектирования и структур данных при

детализации проектного решения;

4 – правильно оформлен код программы, точно соответствующий обоснованному паттерну и выбранным структурам данных;

5 – продемонстрирована работоспособность программы и корректно оформлена пояснительная записка.

1.2. Оформление пояснительной записки

Общий объём пояснительной записки в пределах 20-30 листов машинописного текста, не считая приложений. Для написания текста рекомендуется шрифт Times New Roman 14, через 1,5 интервала, выравнивание по ширине, абзацный отступ 1,25 см (с соблюдением следующих размеров полей: левое – 2,5 см, верхнее – 2 см, нижнее – 2 см, правое – 1 см). Страницы нумеруются внизу в центре, начиная с титульного листа, на котором номер страницы не ставится.

Примерное содержание пояснительной записки к курсовому проекту представлено ниже.

Введение

Раздел 1. Формирование требований к программному продукту и проекту

1.1 Описание предметной области программы

1.2 Анализ существующих решений

1.3 Моделирование вариантов использования

Раздел 2. Проектирование программного средства

2.1 Обоснование выбора архитектуры программы

2.2 Проектирование интерфейса пользователя

2.3 Построение модели данных/Формализация процесса обработки данных/Моделирование состояния системы

Раздел 3. Конструирование программного средства

3.1 Описание классов приложения (детализация диаграммы классов и описание назначения полей и методов каждого класса с обоснованием реализации выбранных паттернов проектирования)

3.2 Моделирование взаимодействия пользователя с программой (диаграммы последовательностей для каждого варианта использования и их краткое описание, тестовые сценарии для каждого варианта использования)

3.3 Развертывание ПС/Интеграция ПС/Верификация ПС

Заключение

Список использованных источников

Приложение 1. Техническое задание или Спецификация ПС

Приложение 2. Диаграмма классов приложения

Приложение 3. Листинг программного модуля

Приложение 4. Руководство пользователя

Естественно, что это исчерпывающее содержание, но обязательно должны быть введение, 2-3 раздела основной части, заключение, список использованных источников и приложения, включающие документацию процесса разработки ПС.

Цифровой материал большого объема выполняется в виде таблиц, которые располагают непосредственно в тексте после первого упоминания. На все таблицы должны быть ссылки в тексте. Номер и название таблицы располагают над ней слева без абзацного отступа.

Пример:

Таблица 1. - Основные характеристики системы

Рисунки входят в общий объём и нумеруются в пределах всей пояснительной записки. Все иллюстрации должны иметь ссылки в тексте. Номер и название помещают ниже рисунка и подрисуночного текста с поясняющими данными.

Пример:

Рисунок 1. – Структурная схема системы.

Сведения об использованных источниках следует располагать в порядке появления ссылок в тексте работы и нумеровать. Сведения об учебниках, методических пособиях и монографиях должны включать автора, название, издательство и год издания. Интернет источники должны включать название материала, автор, полная ссылка на сайт, включающая html-страницу.

Пример:

И.Иоффе Д.С. Стандарт IEEE 1284.
http://www.rusdoc.ru/material/hardware/iee/ieee_1284.htm.

2. Выполнение основных этапов проекта

2.1 Рекомендации по выбору темы

Выбор темы – это очень важный этап, поскольку именно он определяет направление и успех всей последующей работы.

Если тема очень простая, то вам не удастся много «выжать» из нее. Программа неизбежно получится слишком маленькой для того, чтобы претендовать на признание ее в качестве проекта. Вы должны выбрать такую тему, чтобы она (при условии тщательной проработки) была очень продуктивной, чтобы не приходилось что-то искусственно придумывать ради увеличения объема программы.

Выбрать тему курсовой работы вы должны самостоятельно. Конечно, вы можете (и должны) посоветоваться с преподавателем. Курсовой проект – это не типовый расчет, в котором вам для решения предлагаются уже сформулированные задачи. Нужно учиться видеть проблемы в окружающей вас среде и самостоятельно формулировать задачи по решению этих проблем. Никакой преподаватель не в состоянии предложить каждому из вас тему, которая была бы интересной для вас, новой, посильной, перспективной, непохожей на другие темы, и т. п. Поэтому ваше активное участие в выборе темы просто необходимо.

При выборе темы оттолкнитесь от сферы ваших интересов и тех областей деятельности, с которыми вы наиболее часто сталкиваетесь. Выбранное название должно:

- отражать суть работы;
- соответствовать объему работы;
- быть благозвучным;
- быть кратким и емким;
- быть русскоязычным.

Выбранную тему согласуйте с руководителем, но если самостоятельных идей нет, то выберите стандартную тему из табл. 1 применительно к интересной Вам предметной области.

Таблица 1 – Тематика курсового проектирования

Тема	Варианты предметной области	Минимальный функционал продукта
Разработка графического редактора	редактор диаграмм классов, блок-схем устройств, карт помещений и маршрутов	создание, изменение и удаление графических примитивов на рабочей панели; сохранение результатов с последующей загрузкой для редактирования;
Разработка планировщика задач	планирования личного времени, инновационного проекта, путешествия	создание, изменение и удаление списка задач; поиск и ранжирование задачи по заданному критерию;
Разработка управления теплицей	управление лифтом, светофором, контроль заявок на обслуживание	извлечение события из очереди; выполнение заданной последовательности действия по типу события; публикация результата
Разработка метеосервера	сервера обмена сообщениями, учета заявок, управления продажами,	обработка запроса клиента; отправка ответа запросившему клиенту; одновременная работа с несколькими запросами

	совместных покупок	
Разработка пакета программ статистического анализа	ROC-анализ, SWOP-анализ	Выделение специфических данных из файла, применение к данным не менее 3 методов статистической обработки в произвольном порядке; визуальное представление результатов в графической и табличной форме
Разработка компьютерной игры	в карты с компьютером, раскладывание пасьянса, игра на заданном поле	Игровой геймплей, сценарий игры с участием двух персонажей или игрового поля с несколькими активными, наличие не менее 10 различных визуальных эффектов

Проведите согласование темы с научным руководителем для этого разработайте концепцию проекта (табл.2).

Таблица 2 – Концепция проекта

Для	<i>[целевая аудитория покупателей]</i>
который	<i>[описание потребностей или возможностей]</i>
Этот (имя продукта)	<i>является [категория продукта]</i>
который	<i>[ключевое преимущество, основная причина для покупки или использования]</i>
в отличие от	<i>[основной конкурирующий продукт, текущая система или текущий бизнес-процесс]</i>
наш продукт	<i>[его основное преимущество или отличительная особенность]</i>

2.2 Анализ требований и составление внешнего описания

В этом разделе представляется анализ поставленной задачи или проблемы с целью подготовки внешнего описания.

2.2.1 Общее описание программного продукта

На верхнем уровне специфицирования программного продукта обычно разрабатывается **Общие требования к ПС**, которые предназначены для заказчика (Software Specification Document). Это документ, объясняющий в бизнес-терминах, что должна делать система. Документ разрабатывается с ориентацией на пользователя и, соответственно, должен отражать все его интересы. Документ не должен быть перегружен техническими подробностями. Пользователю интересно, какие меню, экраны и отчеты будут представлены в программе, и как программа будет осуществлять переходы из одной точки в другую. В зависимости от уровня подготовленности заказчика спецификация может иметь различную степень детализации.

Пример. Общее описание системы.

Программа «Цифровая логика» предназначена для обучения студентов направления «Информатика и вычислительная техника», изучающих устройство ЭВМ.

Графический редактор позволяет создавать, просматривать, обрабатывать и редактировать цифровые изображения (рисунки, картинки, фотографии) на компьютере.

Особенностью растровых изображений является представление графики как набор точек, т.е. матрицы (bitmap), каждое значение – цвет, соответствующей точки на экране. Растровое изображение – это файл данных или структура, представляющая прямоугольную сетку пикселей. Пиксель – наименьшая единица двумерного цифрового изображения в растровой графике. Пиксель представляет собой неделимый объект прямоугольной (обычно квадратной) формы, обладающий определённым цветом. Растровое компьютерное изображение состоит из пикселей, расположенных по строкам и столбцам матрицы (bitmap).

Растровые графические редакторы позволяют пользователю рисовать и редактировать изображения на экране компьютера, а также сохранять их в различных растровых форматах, таких как, например, JPEG и TIFF, позволяющих хранить растровую графику с незначительным снижением качества за счёт использования алгоритмов сжатия с потерями, PNG и GIF, поддерживающими хорошее сжатие без потерь, и BMP, также поддерживающем сжатие (RLE), но в общем случае представляющем собой несжатое «попиксельное» описание изображения.

Наиболее известные растровые редакторы: GIMP, Adobe Photoshop, Microsoft Paint.

Поскольку разработка программного продукта осуществляется в образовательных целях, то следует обратиться к аналогам, чтобы сформировать четкое представление функций и характеристик программы. Поэтому следующий этап состоит в поиске, испытаниях и анализе программных средств аналогичных разрабатываемому.

Рекомендуется следующая последовательность работ.

1. Список лидеров, на основе профессиональных обзоров, результатов опросов или продаж/скачиваний.

2. Сравнительная характеристика проблем, решаемых конкурентами

Проблема	Сегмент рынка/профиль пользователя	Решение 1	Решение 2	...	Описание	Приоритет
	*	+/-				e/u/d

*) указывается целевой рынок пользователей (U – обычных, P – «продвинутых») или корпоративных (XS – компании до 10 сотрудников, S – 10-25, M – 25 -250, L - 250-2500, C – более 2500 сотрудников)

+/-) отмечается наличие или отсутствие указанной возможности;

e/u/d) устанавливается насколько важна данная характеристика ПП (обязательно (essential), полезно (useful), желательно (desirable)).

3. Список возможностей, которые были реализованы в конкурентных продуктах для удовлетворения проблем, описанных в предыдущем пункте (может быть оформлено в виде таблицы с указанием «+», если продукт предоставляет эту возможность, или нет – «-».)

4. Резюме по лучшим программам-конкурентам с описанием интересных

идей, решений.

2.2.2 Функциональные требования

На следующем этапе разработчик должен определить все функциональные требования, включая передаваемые параметры, глобальные структуры и переменные, вызываемые подпрограммы и т.д.

Пример. Требования к функциональным характеристикам системы контроля успеваемости.

Система должна обеспечивать возможность выполнения следующих функций:

инициализацию системы (ввод списка группы и т.п.);

ввод и коррекцию текущей информации о ходе выполнения учебного графика конкретным студентом;

хранение информации в течение длительного времени;

получение сведений о текущем состоянии выполнения учебного графика студентами в следующих вариантах:

а) процент успеваемости по конкретному студенту по всем предметам;

б) процент успеваемости по всем студентам по конкретному заданию;

в) список студентов, не сдавших конкретное задание.

Исходные данные:

список студентов группы;

перечень предметов, перечень заданий по каждому предмету и сроки их сдачи;

текущие данные (еженедельно): процент выполнения каждым студентом каждого задания учебного графика.

Для этой цели следует использовать прецеденты (пример графического представления прецедента диаграммой вариантов использования на рис. 1.

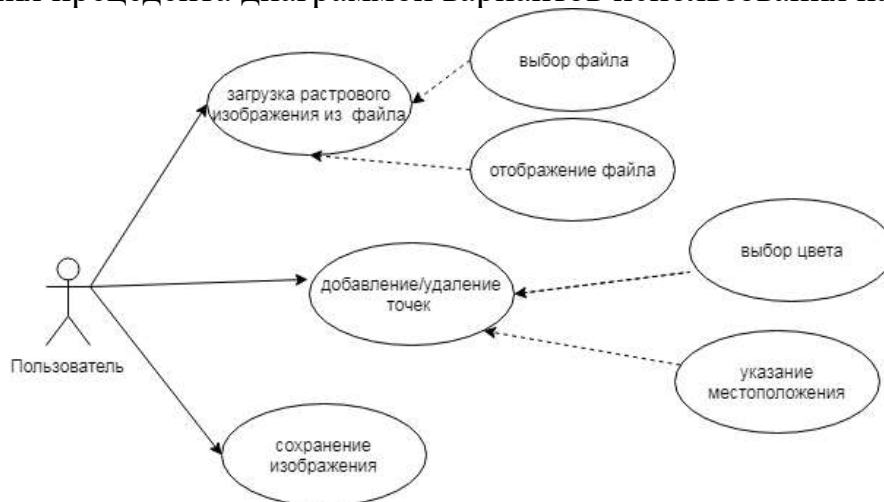


Рис. 1. - Ключевые варианты использования

Прецеденты начинают описание того, что будет делать система. Однако для реального проектирования системы потребуются более подробные данные, которые отражены в спецификациях прецедентов. Спецификация прецедента должна подробно определять то, что будут делать с системой пользователи, и

то, что делает сама система.

Несмотря на детализацию, спецификация прецедента остается независимой от реализации. Целью такой спецификации является описание действий системы, а не того, *как* выполняются эти действия. Обычно спецификация содержит:

- *Краткое описание*
- *Действующие лица*
- *Предусловия (Preconditions)*
- *Основной поток событий*
- *Альтернативный поток событий*
- *Постусловия (Postconditions)*

Любой прецедент должен иметь краткое описание, объясняющее действия в этом прецеденте. Описание должно быть кратким, но в него необходимо включить сведения о разных типах пользователей, выполняющих данный прецедент, и ожидаемый результат. Во время работы над проектом (особенно если проект сложный) эти описания будут напоминать членам команды, почему тот или иной прецедент был включен в проект и что он должен делать. Четко документируя таким образом цели каждого прецедента, можно уменьшить неразбериху, возникающую среди разработчиков.

В *предусловиях* прецедента перечислены все условия, который должны быть выполнены *прежде, чем данный прецедент начнет работать*. Не у всех прецедентов есть предусловия.

Диаграммы прецедентов не предназначены для показа последовательности исполнения прецедентов, но предусловия позволяют документировать подобную информацию, например, необходимость последовательного выполнения двух прецедентов.

Конкретные детали прецедентов отражены в основном и альтернативном потоках событий. Поток событий — это поэтапное описание того, что происходит во время выполнения прецедента. Поток событий уделяет внимание тому, **что делает система**, причем описывает это **с точки зрения пользователя**. Основной и альтернативный потоки событий содержат:

- Процедуру начала прецедента
- Различные этапы исполнения прецедента
- Нормальный (основной) поток событий в прецеденте
- Все отклонения от основного потока событий, которые называются альтернативными потоками событий
- Все потоки ошибок
- Процедуру завершения прецедента

Помимо описания потоков событий в текстовом виде часто используются диаграммы деятельности, таблицы типа воздействие-отклик, графы переходов и состояний.

2.2.3 Нефункциональные требования

На последнем этапе определяются нефункциональные требования. Многие требования не являются функциональными, и описывают только атрибуты системы или признаки окружения системы.

Есть много различных видов требований. Один из способов их категоризации описывается как модель FURPS+, использующая акроним, чтобы описать основные категории и подкатегории требований, как показано ниже.

- Функциональность (Functionality),
- Удобство использования (Usability),
- Надежность (Reliability),
- Производительность (Performance),
- Поддерживаемость (Supportability)

"+" помогает Вам также не забыть включать такие требования, как

- конструктивные ограничения,
- требования к интерфейсам,
- физические требования.

Функциональные требования могут включать в себя:

- наборы функций,
- возможности,
- безопасность.

Требования удобства использования могут включать такие подкатегории, как человеческий фактор, эстетика, последовательность пользовательского интерфейса, он-лайн и контекстно-зависимая справка, мастера и агенты, пользовательская документация, учебные материалы.

Требования к надежности могут быть описаны как доступность (в процентах от имеющегося времени, часов эксплуатации, техническому обслуживанию доступа, ...), частота / степень интенсивности отказов, восстанавливаемость, предсказуемость, точность, среднее время наработки на отказ (MTBF).

Требования к производительности накладывают определённые условия на одно из функциональных требований. Так для указанного действия, они могут выражаться в виде пропускной способности (например, транзакций в секунду), времени отклика, времени восстановления, использования ресурсов (память, диск, процессор, ...).

Требования к сопровождаемости (поддерживаемости) могут включать тестируемость, расширяемость, адаптивность, ремонтпригодность, совместимость, конфигурируемость, удобство обслуживания, особенности установки, локализацию (интернационализацию).

Конструктивные требования определяют ограничения дизайна, указывают или ограничивают конструкцию системы. В этом пункте должны быть указаны любые ограничения проектирования в строящейся системе. Ограничения дизайна представляют собой конструктивные решения, которые введены и должны соблюдаться. Например, языки программирования, требования к процессу разработки программного обеспечения, предписанного использования

инструментов разработки, архитектурные и конструктивные ограничения, покупные комплекты, библиотеки классов и т.д.

Требования к интерфейсам определяют интерфейсы, которые должны поддерживаться приложением. Их можно разделить на:

- пользовательский интерфейс (пользовательские интерфейсы, которые должны быть реализованы с помощью программного обеспечения);
- аппаратный интерфейс (аппаратные интерфейсы, которые должны поддерживаться программным обеспечением, в том числе логическую структуру, физические адреса и ожидаемое поведение, и т.д.);
- программный интерфейс (программные интерфейсы для других компонентов системы. Они могут быть приобретенными, повторно используемыми компонентами из другого приложения, компонентами разрабатываемыми для подсистемы, которая выходит за рамки этого проекта, но с которой это приложение должно взаимодействовать).

Пример. Требования к надежности.

1. *Предусмотреть контроль входных значений в диапазоне от 1 до 10.*
2. *Обеспечить сохранение результатов предыдущей корректировки файла данных в файле с расширением .bak.*

2.3. Проектирование структуры и компонентов программного средства

В этом разделе описывается проектирование компонентов в соответствии с выбранной архитектурой.

Архитектура – это структура, взаимодействие и управление. Начинать проектирование следует со структуры, поэтому на первом этапе необходимо определиться с функциональностью. Деление на модули/подсистемы лучше всего производить исходя из тех задач, которые решает система. Основная задача разбивается на составляющие ее подзадачи, которые могут выполняться независимо друг от друга. Каждый модуль должен отвечать за решение какой-то подзадачи и выполнять соответствующую ей функцию.

В большинстве программных систем можно выделить три группы функций, ориентированных на решение различных подзадач:

1. функции ввода и отображения данных (обеспечивают взаимодействие с пользователем);
2. прикладные функции, характерные для данной предметной области;
3. функции управления ресурсами (файлами, базой данных и т.д.)

Выполнение этих функций в основном обеспечивается программными средствами, которые можно представить в виде взаимосвязанных компонентов (исполняемый код, имеющий API) или связанных программных модулей.

Наиболее часто используемые программные архитектуры представлены на рис. 2, пример модульного представления архитектуры на рис. 3.

Обратите внимание что каждый модуль или компонент должен быть описан (назначение и функции в системе), декомпозирован до классов приложения для его реализации. Как правило, название пакета отражает функционал выде-

ленного программного модуля. Для большинства программ подходит архитектурный стиль MVC (Model-View-Controller), на рис. 4 представлена обобщенная схема взаимодействия компонентов MVC.

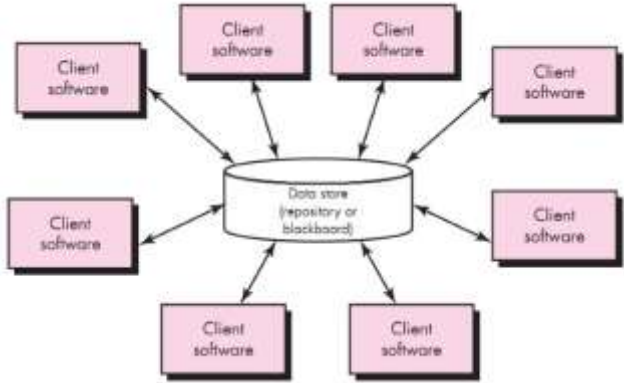
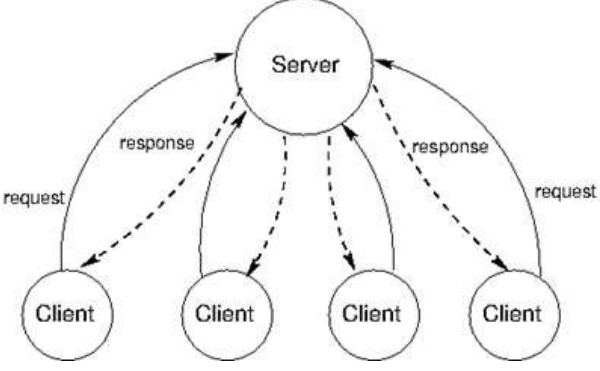
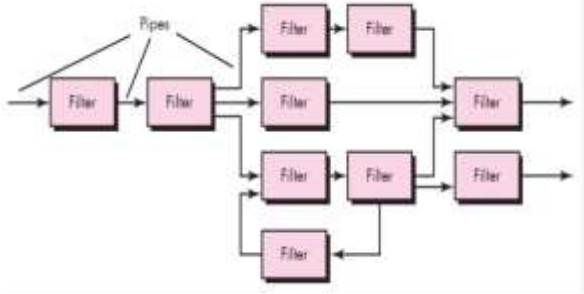
	
Архитектура с общим репозиторием, который обеспечивает обмен данными – один записал, другой прочитал	Клиент-серверная, обеспечивает взаимодействие запрос-ответ между двумя видами компонентов через среду обмена сообщениями
	
Многоуровневая, где каждый слой выполняет определённую функцию и доступ только через предшествующий	Каналы передачи данных и фильтры (их обработчики) составляют процесс обработки данных

Рис. 2 – Шаргалка по выбору архитектуры решения



Рис. 3 - Архитектура системы хранения личных достижений

Компонент Модель(Model) представляет собой совокупную модель данных предметной области, реализованной в приложении. Представление(View) – это компонент, отвечающий за отображение Модели и реализацию ее взаимодействия с пользователем. Вся логика по обработке действий пользователя через View для управления Model сосредоточена в компоненте Контроллер (Controller).



Рис. 4 - Архитектура приложения в стиле MVC

Пример обработки графических данных с использованием архитектуры «Каналы и фильтры» на рис. 5

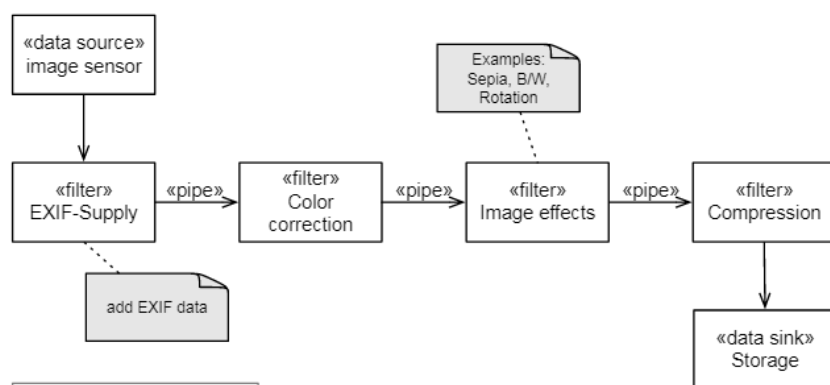


Рис.5 – Пример архитектура приложения с обработкой данных

2.4. Разработка пользовательского интерфейса

Интерфейс имеет важное значение для любой программной системы и является неотъемлемой ее составляющей, ориентированной, прежде всего, на конечного пользователя. Именно через интерфейс пользователь судит о прикладной программе в целом; более того, часто решение об использовании прикладной программы пользователь принимает по тому, насколько ему удобен и понятен пользовательский интерфейс. Разработка графического пользовательского интерфейса для некоторых программ – обучающих, игровых, имитаторов устройств и приборов, выступает как отдельный компонент создаваемого ПО и влияет на весь процесс проектирования.

Этот раздел должен начинаться с обзора различных способов и форм взаимодействия пользователя с системой и обоснования выбора определенной формы диалога (лежащего в основе любого взаимодействия) для общения с разрабатываемым программным продуктом.

Задачами данного взаимодействия является передача информации (исходных данных) от пользователя прикладной программе, выходных данных (результатов работы программы) пользователю. В соответствии с принятыми сейчас стандартами функцией интерфейса является также объяснение результатов работы прикладной программы, включая диагностику ошибок.

В состав пользовательского интерфейса входят:

- базисная система сообщений (система понятий предметной области);
- система сообщений для пользователя;
- система сообщений для прикладной программы;
- средства обеспечения удобства и комфорта работы пользователя;
- средства интеллектуальной поддержки пользователя;
- средства управления взаимодействием пользователя и интерфейса.

Таблица 3 – Шпаргалка по выбору интерфейса пользователя

Стиль взаимодействия	Основные преимущества	Основные недостатки	Примеры приложений
Прямое манипулирование	Быстрое и интуитивно понятное взаимодействие	Сложная реализация. Подходит только там, где есть зрительный образ задач и объектов	Видеоигры, системы автоматического проектирования
Выбор из меню	Сокращение количества ошибок пользователя. Минимальный ввод с клавиатуры.	Медленный вариант для опытных пользователей. Усложнен, если меню состоит из большого количества вложенных пунктов.	Главным образом системы общего назначения
Заполнение форм	Простой ввод данных. Легок в изучении	Занимает пространство на экране.	Системы управления запасами, обработка финансовой информации.
Командный язык	Мощный и гибкий	Труден в изучении. Сложно предотвратить ошибки ввода.	ОС, библиотечный системы.
Естественный язык	Подходит неопытным пользователям. Легко настраивается.	Требуется большого ручного набора	Системы расписания, системы хранения данных WWW

При разработке ПС должна определяться структура диалога и приводиться диаграмма диалога интерфейса, отражающая эту структуру. Кроме того, определяется набор необходимых форм и строится граф или диаграмма состояний интерфейса (рис. 6).

В случае табличной формы диалога производится описание всех оконных форм и меню (рис. 7). В случае использования директивной или фразовой формы описываются основные команды, в форме таблицы (пример в табл. 4).

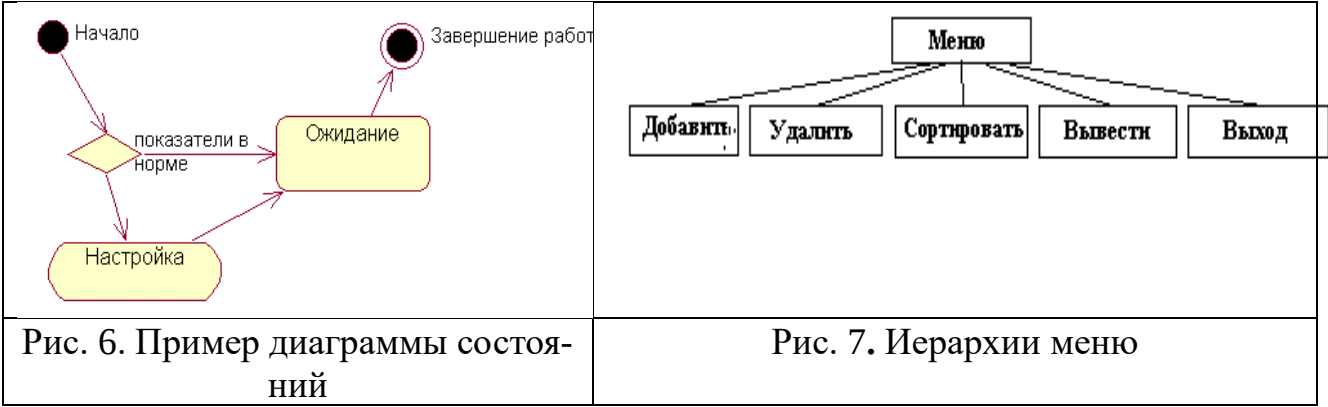


Таблица 4 – Представление пользовательского интерфейса

Действие пользователя	Отклик системы	Элемент интерфейса
Нажатие на кнопку Старт	Вывод формы для ввода логина и пароля	Панель основной сцены

При использовании событийного программирования необходимо разработать графический интерфейс пользователя, провести компоновку панелей в сцену и элементов управления. Представим «раскадровку» рабочих окон по шаблону на рис. 8.

Цель экрана	Основное окно приложения	
Реализация		
Элемент управления	Действие пользователя	Отображаемый результат
Поле ввода с меткой «Компьютер»	ввод числовых значений	В поле отображается введенное значение

Рис. 8. Раскадровка пользовательского интерфейса

Следует также группировать однотипные элементы с одной функциональностью на панелях и в других контейнерах и ссылаться в описаниях на их названия. Раскадровка может быть дополнена иерархией компонентов.

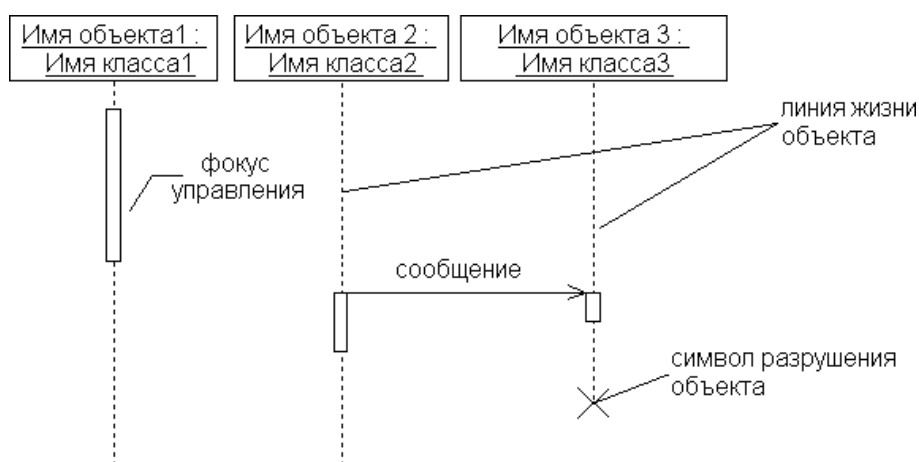
После уточнения интерфейса выполняется декомпозиция предметной области задачи в соответствии с выбранной архитектурой, т. е. создается структура классов программного средства.

2.5 Моделирование сценариев использования

Для 3-5 основных сценариев работы программы в соответствии с ранее разработанной диаграммой прецедентов моделируется последовательность взаимодействия программных объектов. Эти диаграммы, нужны **НЕ** для отображения взаимодействия пользователя с интерфейсом (так хороший интерфейс должен быть интуитивно понятным), а для последующей реализации программы.

Диаграмма последовательностей изображается как таблица, в которой по оси X представлены объекты, а по оси Y - сообщения, упорядоченные по времени.

Построение диаграммы начинается с размещения объектов, участвующих в реализуемом сценарии работы программы, в верхней части диаграммы. От них вертикально располагают пунктирные линии, символизирующие время жизни объекта (рис. 9).



Сообщения - наносится в виде стрелки, направленной от одной линии жизни к другой. Стрелка указывает на приемник сообщения.



синхронные требуют возврата ответа,

асинхронные – ответ не требуется, и вызывающий объект может продолжать работу

возврат ответа на сообщение - на синхронное сообщение (не обязателен на схеме, но может использовать для демонстрации возвращаемого значения)

Рис. 9 – Элементы диаграммы последовательности

Следует наносить сообщения, начиная от инициатора воздействия, располагая все последующие сообщения сверху вниз между линиями жизни, показывая свойства этого сообщения (сигнатуру метода, передаваемые параметры), см. шаблон на рис. 10.

Для каждого сценария, детализируется последовательность взаимодействия.

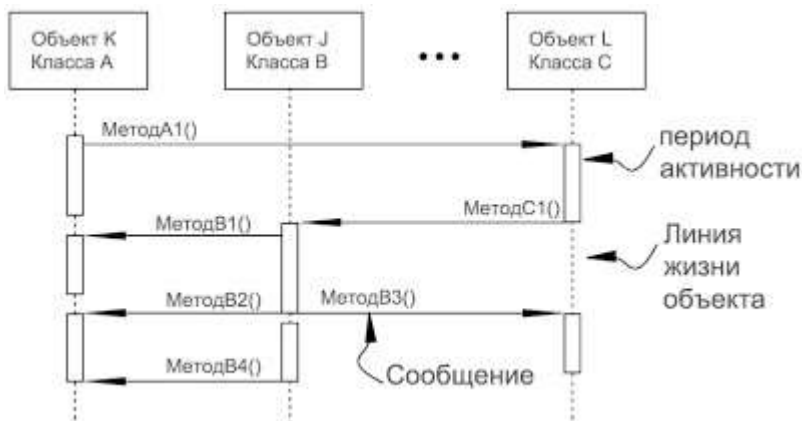


Рис. 10 – Шаблон построения диаграммы последовательностей взаимодействия объектов программы

2.6. Конструирование программы

Для ПС средней сложности описание реализации сводится в его представлении в виде различных артефактов проектирования и конструирования. Следует обязательно указать:

- 1) язык, среду разработки и другие инструментальные средства использованные при создании ПП;
- 2) структуру программного проекта (рис. 11);
- 3) краткое описание каждого пакета или указание какому функциональному модулю они соответствуют;
- 4) пакет модели или бизнес-логики представляется развернутой диаграммой классов с описанием полей и методов средствами JavaDoc;

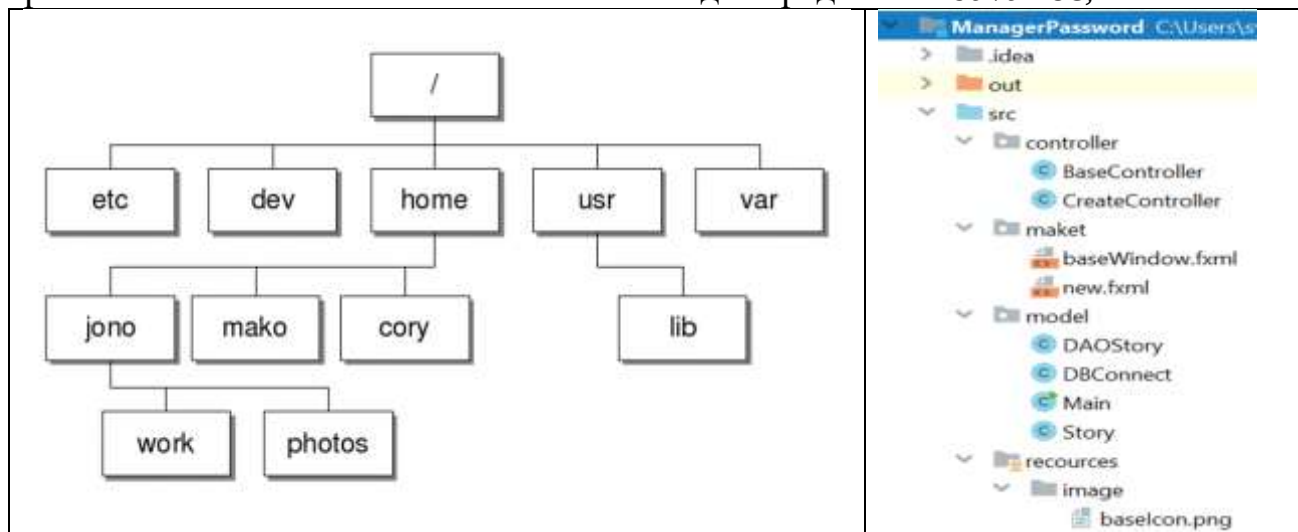


Рис. 11 – Примеры представления структуры проекта

В описании реализации должно быть включено пояснение использования методов, определяющих следующие принципы конструирования.

1. Параллелизм (Concurrency), включая подходы и методы организации процессов, задач и потоков для обеспечения эффективности, атомарности, синхронизации и распределения (по времени) обработки информации.

2. Контроль и обработка событий (Control and Handling of Events). Хорошей практикой является использование неявных методов обработки событий, часто реализуемых в виде функции обратного вызова (callback), как одной из фундаментальных концепций обработки событий.

3. Обработка ошибок и исключительных ситуаций и обеспечение отказоустойчивости (Errors and Exception Handling , Fault Tolerance)

4. Использование паттернов проектирования.

Шаблоны (паттерны) проектирования — это проверенные и готовые к использованию решения часто возникающих в повседневном программировании задач. В общем смысле паттерн представляет собой образец решения некоторой задачи так, что это решение можно использовать в различных ситуациях.

В общем случае каждый паттерн состоит из таких составляющих:

1. Имя является уникальным идентификатором паттерна. Имена паттернов проектирования Компоновщик, Декоратор, Адаптер, Одиночка, Фабрика, Прототип, Строитель являются общепринятыми.

2. Задача описывает ситуацию, в которой можно применять паттерн.

3. Решения задачи проектирования в виде паттерна определяет общие функции каждого элемента дизайна и отношения между ними.

4. Результаты представляют следствия применения паттерна.

При реализации следует тщательно подходить к выбору паттернов, чтобы чрезмерно не усложнять код, при этом в представленных ниже случаях использование шаблонных решений необходимо.

Зачастую, такой подход может привести к правильному, с академической точки зрения, но не отвечающему требованиям гибкости и масштабируемости решению. Использование паттернов дает общую терминологию для описания решения (ссылки на паттерны проектирования, как на известные решения задач, можно добавлять в проектную документацию).

2.7 Верификация ПО

Требования, подлежащие проверке при испытании программы, а также порядок и методика их контроля должны быть представлены в ТЗ, в частности приводятся тестовые наборы и ожидаемые результаты.

Особое внимание следует уделять созданию тестовой документации, которая может быть включена в отдельное приложение, а результаты тестирования представляются в виде чек-листа в приложении.

В общем случае, тестовая документация может содержать: заголовок, пошаговое описание, ожидаемый результат, критерий соответствия ожидаемого результата фактическому (как узнать, что мы получили правильный результат).

Пример 1. Проверка того, что программа умеет показывать файлы формата BMP

Шаг 1. Нажать кнопку "Выбрать файл"

Шаг 2. Выбрать файл с расширением BMP

Шаг 3. Нажать кнопку "Открыть"

Ожидаемый результат: содержимое файла показано в графическом виде, в полноэкранном режиме.

Таблица 5 – Пример TestCase1.1 для проверки сценария рисования

Название:	Тест: рисование по точкам	
Функция:	Рисование заданным цветом при движении мыши с удержанием левой кнопки	
Действие	Ожидаемый результат	Результат теста: • пройден • провален • заблокирован
Предусловие:		
Очистите панель для рисования	Белое поле панели рисования	
Выберите красный цвет на элементе управления Выбор цвета	Щелчок по кнопке. Выбор на палитре красного цвета. Щелчок ОК.	
Шаги теста:		
Выберите точку на панели. Щелкните мышью. Не отпуская кнопки, двигайте мышь.	Точки красного цвета повторяют путь движения мыши.	
Постусловие:		
Отпустите мышь	Точки красного цвета повторяющие путь движения мыши.	

Тестовая документация разрабатывается на этапе спецификации программного продукта, и используется при отладке. Поэтому должна быть включена в раздел верификации ПО. По возможности прилагается скриншот решения контрольного примера.

3. Составление пояснительной записки

3.1. Рекомендации по написанию введения и заключения

Введение и заключение не должно быть большим, вполне достаточно одной страницы для того, чтобы отразить основные вопросы.

Во введении необходимо отразить:

1. Актуальность выбранной вами темы.
2. Цель работы и задачи, которые следует решить, чтобы ее достигнуть.

Тема актуальна – это значит, что она соответствует требованиям сегодняшнего дня. Она укладывается в русло основных тенденций развития информационных технологий. При этом актуальность темы учебной работы, которой является курсовой проект, можно рассматривать не только с какой-то всеобщей, глобальной позиции, но и с точки зрения вашего профессионального роста. Поэтому возможно, что для вас в настоящий момент более актуальной будет не та технология, о которой сейчас говорить весь компьютерный мир, а какая-то менее яркая технология или область знаний, которая лежит в основе этой суперсовременной технологии. Поэтому реализация выбранной вами темы сможет помочь вам на следующем этапе изучения информационных технологий, будет способствовать становлению вас как специалиста.

От актуальности зависит цель, которую вы ставите перед собой как результат выполнения курсового проекта

Целью вашего курсового проекта может быть, например, следующее: создать технологическую базу для разработки программного продукта «А», использование которого в сфере «Б» позволит повысить эффективность выполнения операций типа «В».

Принято также формулировать и ряд задач, решение которых как раз и служит достижению поставленной цели. Задачами, решение которых позволит достичь поставленной цели, могут быть следующие:

1. Изучить основные конструкции языков программирования *Perl* и *JavaScript*, познакомиться с основными возможностями системы управления базами данных *PostgreSQL*, познакомиться с технологией *Ajax*.

2. Проанализировать несколько существующих программных продуктов, построенных на основе этого инструментария.

3. Спроектировать и реализовать собственный программный продукт (например, информационную систему «Домашняя библиотека») с применением указанных технологий.

4. Провести тестирование и внедрение (если возможно, конечно) программного продукта в опытную эксплуатацию.

При подведении итогов выполнения курсового проекта в разделе «Заключение» необходимо показать, что поставленные задачи решены и цель достигнута.

3.2 Составление программной документации

3.2.1 Написание технического задания

Техническое задание – это документ, определяющий цели, требования и основные исходные данные, необходимые для разработки автоматизированной системы управления.

При разработке технического задания необходимо решить следующие задачи:

- установить общую цель создания ИС, определить состав подсистем и функциональных задач;
- разработать и обосновать требования, предъявляемые к подсистемам;
- разработать и обосновать требования, предъявляемые к информационной базе, математическому и программному обеспечению, комплексу технических средств (включая средства связи и передачи данных);
- установить общие требования к проектируемой системе;
- определить перечень задач создания системы и исполнителей;
- определить этапы создания системы и сроки их выполнения;
- провести предварительный расчет затрат на создание системы и определить уровень экономической эффективности ее внедрения.

Типовые требования к составу и содержанию технического задания на разработку программного продукта приведены в шаблоне (приложение 4).

3.2.2 Разработка пользовательской документации

Документация на программный продукт включает:

- а) описание программы;
- б) руководство пользователя;
- в) справочник по применению;
- г) руководство по управлению ПС.

3.2.2.1 Описание программы

Описание программы обязательно должно включать информационную часть – аннотацию и содержание. Основная часть документа должна состоять из вводной части и следующих разделов:

- функциональное назначение;
- описание логики.
- условия применения;
- состав и функции.

В зависимости от особенностей программы допускается введение дополнительных разделов.

В **вводной части документа** приводится информация общего характера о программе – полное наименование, обозначение, ее возможные применения и т. п.,

Пример. Программа «Оценка индекса массы тела» предназначена для расчета индекса массы тела и получения оценки веса тела по заданным шка-

лам.

Программа реализована в рамках проекта «Инновационный потенциал средней школы»

Программа поддерживает две шкалы оценки веса – рекомендованную ВОЗ и МинСоцРазвития Российской Федерации, а также предусмотрена возможность включения новых критериев оценки по специально разработанным шаблонам.

В разделе «**Функциональное назначение**» указывают назначение программы и приводят общее описание функционирования программы, ее основные характеристики, сведения об ограничениях, накладываемых на область применения программы, а также указывают типы электронных вычислительных машин и устройств, которые используются при работе.

Пример: Программа предназначена для решения задач ... Программа представляет собой ядро автоматизированного рабочего места ...

Пользователь имеет возможность ..., осуществить ..., запустить ..., проанализировать ..., получить результаты анализа и обработки ..., построить ... и т. п.

В разделе «**Описание логики**» приводят:

- описание структуры программы и ее основных частей

Пример. В состав программы входит следующее:

- пользовательский интерфейс,
- модуль определения путей в графе,
- модуль расчета передаточной функции,
- текстовый редактор.

- описание функций составных частей и связей между ним,

Пример. Программа состоит из шести модулей: интерфейсный модуль; модуль определения ...; модуль расчета ...; модуль ...и т. п.

Интерфейсный модуль построен на двух типах диалогов: диалог «вопрос – ответ» и диалог типа «меню». Интерфейсный модуль управляет ...

Модуль определения ... Он является ...

Модуль расчета ...и т. д.;

- сведения о языке программирования, среде разработке и прочее;
- описание входных и выходных данных для каждой из составных частей,

Пример.

ВХОДНЫЕ ДАННЫЕ. Входными данными для программы является текстовый файл, описывающий расширенную матрицу инцидентий графа исследуемой системы.

ВЫХОДНЫЕ ДАННЫЕ. Выходными данными являются:

- выводимая на экран графическая и текстовая информация (результаты анализа системы);
- файлы в одном из графических форматов – копии изображения построенных характеристик (АЧХ, ФЧХ и т. д.);
- текстовые файлы – отчеты о проведенных исследованиях;

– диагностика состояния системы и сообщения о всех возникших ошибках;

- описание логики составных частей (при необходимости следует составлять описание схем программ). При описании логики программы необходима, естественно, привязка к тексту программы.

В разделе **«Состав и функции»** указывают описание состава и функции программ, применяемых методов решения задач.

В разделе **«Условия применения»** указывают условия, необходимые для выполнения программы (требования к необходимым для данной программы техническим средствам и другим программам, общие характеристики входной и выходной информации, а также требования и условия организационного, технического и технологического характера и т. п.).

Пример. Технические характеристики компьютера: Intel Core i-5 и выше; ОЗУ не менее 8 Гб; свободное место на диске не менее 500 Мб. Дополнительных периферийных компонентов не требуется.

При необходимости следует указать имя загрузочного модуля, а также описание всей процедуры «Вызова и загрузки системы»,

Пример. Загрузка программы осуществляется по имени загрузочного модуля – SBM80N.EXE с возможным указанием имени файла данных. В приложение к описанию могут быть включены справочные материалы (иллюстрации, таблицы, графики, примеры и т. п.).

3.2.2.2 Руководства пользователя

Существует множество видов предоставления справочной информации пользователю – это и FAQ (frequently asked questions, часто задаваемые вопросы) и онлайн справка и руководство пользователя (user guide) и подсказки (tints), обучающие видео и другие. Выбор представления зависит от назначения и целевой аудитории программы.

Справочник или руководство пользователя включают разделы аналогичные описанию программы.

- Аннотацию, в которой приводится краткое изложение содержимого документа и его назначение. Также рекомендуется писать краткую аннотацию в начале каждого крупного раздела.
- Введение, содержащее информацию о том, как лучше всего использовать данное руководство.
- Содержание.
- Разделы, описывающие, как использовать ПО (глава может соответствовать пользовательской истории или сценарию использования программы).
- Глоссарий и Предметный указатель.

«Описание операций» – это основной раздел документа Руководства пользователя, который содержит пошаговую инструкцию для выполнения того или иного действия пользователем и включает:

- описание выполняемой (выполнения) функции;
- реакцию программы на команды или действия пользователей для вы-

полнения функции.

При описании следует руководствоваться схемой - «действие (пользователя) - результат (ответ программы)». **Руководство пользователя, являясь документом техническим, не преследует цели завоевать расположение читателя. «Любые описания последовательности действий даются в отстраненно-вежливой форме повелительного наклонения».**

Примерный шаблон. Для выполнения операции *ТАКОЙ-ТО* (или «указанной операции») следует:

- *открыть меню раздела (рисунок, поясняющий результат);*
- *выбрать пункт меню Connect target(рисунок, поясняющий результат);*
- *нажать кнопку Get data для получения данных (рисунок, поясняющий результат).*

Раздел «Подготовка системы к работе» должен содержать пошаговую инструкцию для запуска приложения. К этапу подготовки системы к работе можно отнести установку дополнительных приложений (при необходимости), идентификацию, аутентификацию и т.п.

Раздел «Аварийные ситуации» должен содержать пошаговые инструкции действий пользователя в случае отказа работы Системы. Если к пользователю не были предъявлены особые требования по администрированию операционной системы и т.п., то можно ограничиться фразой «При отказе или сбое в работе ПС необходимо обратиться к Системному администратору».

Следует подчеркнуть, что «Руководство пользователя» может представлять собой как краткий справочник по основному функционалу программы, так и полное учебное пособие. Методика изложения материала в данном случае будет зависеть от объема самой программы и требований заказчика.

Чем подробнее будут описаны действия с системой, тем меньше вопросов возникнет у пользователя. Для более легкого понимания всех принципов работы с программой, стандартами в документе Руководство пользователя допускается использовать схемы, таблицы, иллюстрации с изображением экранных форм.

3.3 Рекомендации по оформлению кода

Требования к оформлению текста программы достаточно просты и естественны для грамотного программиста (актуальный стандарт Java Code Conventions). Основное, чем требуется руководствоваться при создании этого документа, – это то, что текст программы должен быть удобочитаемым.

Основная часть документа должна состоять из текстов одного или нескольких разделов, которым даны наименования. Текст каждого программного файла начинается с «шапки», в которой указывают:

- наименование программы,
- автор,
- дата создания программы,
- номер версии,
- дата последней модификации.

Обязательны комментарии, а также строгое соблюдение правил отступа.

Всегда выбирайте имена переменных и функций, имеющие смысловую нагрузку. Так имя TForm1 указывает на поспешность разработки.

Помните, оправдать можно даже неумение создавать программную документацию. А некрасивый текст программы – никогда. Ссылки на то, что этот текст понятен самому автору, всерьез не воспринимаются. Поэтому настройте форматирование кода в среде разработки и используйте возможности, предоставляемые IDE. Так (на примере для Java) специальные комментарии в исходном коде Java, разделенные `/** ... */` разделителями (документирующие комментарии) обрабатываются инструментом Javadoc для создания документации API.

Комментарий документа пишется на HTML и должен предшествовать объявлению класса, поля, конструктора или метода. Он состоит из текстового описания и, следующих за ним, блочных тегов.

```
/**
 * Класс, демонстрирующий применение документирующих комментариев.
 * <p>
 * Создан для методических указаний по курсовому проектированию
 *
 * @author Minakova
 * @version 1.2
 */
public class SquareNum {
    /**
     * Этот метод возвращает квадрат значения параметра num.
     * Это описание состоит из нескольких строк. Число строк
     не ограничивается.
     *
     * @param num Значение, которое требуется возвести в квадрат.
     * @return Квадрат числового значения параметра num.
     */
    public double square(double num) {
        return num * num;
    }
    /**
     * Этот метод получает значение, введенное пользователем.
     * @return Введенное значение типа double.
     * @exception IOException Ошибка ввода.
     * @see IOException
     */
    public double getNumber() throws IOException {
        // создать поток BufferedReader из стандартного потока System.in.
        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader inData = new BufferedReader(isr);
        String str = inData.readLine ();
        return (new Double(str)).doubleValue() ;
    }
}
```

Например в документации API Java рекомендуются следующие правила

составления документирующих комментариев.

Первая строка содержит разделитель начала комментария (/**).

Напишите первое предложение как краткое описание метода, поскольку Javadoc автоматически его индексирует до первой точки.

Если у вас есть более одного абзаца в комментарии к документу, разделите абзацы тегом абзаца<p>.

Вставьте пустую строку комментария между описанием и списком тегов.

Первая строка тегов, начинающаяся с символа «@», завершает описание. (На комментарий к документу приходится только один блок описания, поэтому нельзя продолжать описание после блочных тегов.

Используйте стиль <code> для выделения ключевых слов и имен.

Добавляйте ссылки на уже указанные имена API с помощью тега {@link}, но не злоупотребляйте такими ссылками без необходимости.

Когда вы ссылаетесь на метод или конструктор, который имеет несколько форм указываете только его имя, а если на конкретную форму, используйте круглые скобки и типы аргументов.

Можно использовать фразы вместо полных предложений в интересах краткости.

В описаниях классов/интерфейсов/полей может опускаться субъект и просто указываться объект (например, вместо “Это исключение генерируется при ошибке ввода” исп. “ошибке ввода”)

Описание метода начинается с глагольной фразы типа «делает то-то».

Последняя строка содержит разделитель конца комментария (*/). содержит только одну звездочку.

Наиболее распространение теги в таблице 6, указаны в порядке их включения в комментарий. По умолчанию документируются элементы public и protected. Но это как и другие параметры настраивается в среде разработки перед генерацией документации (рис. 13)

Таблица 6. – Теги документирующий комментариев

	Дескриптор	Описание
1	@author	Обозначает автора программы
2	@version	Обозначает версию класса
3	@param	Документирует параметр метода
4	@return	Документирует значение, возвращаемое методом
5	@exception	Обозначает исключение, генерируемое методом
6	@see	Указывает ссылку на другую тему
7	@since	Обозначает версию, в которой были внесены определенные изменения
8	@deprecated	Указывает на то, что элемент программы не рекомендован к применению

Обратите внимание, что документируется уже реализованный код, т.е. описано то, что реализовано.

Заключение

В учебно-методической разработке представлено описание процесса разработки программного продукта – извлечение и документирование требований, проектирование и реализация приложения. Для каждого этапа указаны необходимые артефакты разработки и представлены шаблоны и примеры для их создания.

Выполнение указанных работ позволит студенту получить навыки создания программного продукта и практически закрепить теоретические сведения, полученные в ходе изучения программной инженерии, последовательно пройдя все этапы создания программного продукта с документированием каждого этапа.

Содержание

ВВЕДЕНИЕ	5
1. Организация процесса выполнения курсового проекта.....	6
1.1. Порядок выполнения проекта.....	6
1.2. Оформление пояснительной записки.....	8
2. Выполнение основных этапов проекта.....	10
2.1 Рекомендации по выбору темы.....	10
2.2 Анализ требований и составление внешнего описания	11
2.2.1 Общее описание программного продукта	11
2.2.2 Функциональные требования	13
2.2.3 Нефункциональные требования	15
2.3. Проектирование структуры и компонентов программного средства.....	16
2.4. Разработка пользовательского интерфейса.....	18
2.5 Моделирование сценариев использования	21
2.6. Конструирование программы	22
2.7 Верификация ПО.....	23
3. Составление пояснительной записки	25
3.1. Рекомендации по написанию введения и заключения.....	25
3.2 Составление программной документации.....	26
3.2.1 Написание технического задания	26
3.2.2 Разработка пользовательской документации.....	26
3.3 Рекомендации по оформлению кода.....	29
Заключение	33

**МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ ПО КУРСОВОМУ ПРОЕКТИРОВАНИЮ
ПО ДИСЦИПЛИНЕ
«ТЕХНОЛОГИЯ ПРОГРАММИРОВАНИЯ»**

*для обучающихся направления 09.03.02 «Информационные системы и технологии»
всех форм обучения*

Составители:

Минакова Ольга Владимировна

Подписано в печать 04.06.2020 Формат 60х84 1/8 Бумага для множительных аппаратов
Уч.-изд. л. 3,3 Усл. печ. л. 3,0.

ФГБОУ ВО «Воронежский государственный технический университет»

396026 Воронеж, Московский просп., 14

Участок оперативной полиграфии издательства ВГТУ

396026 Воронеж, Московский просп., 14